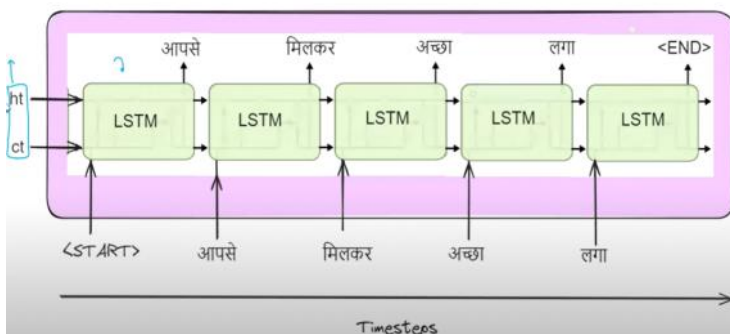
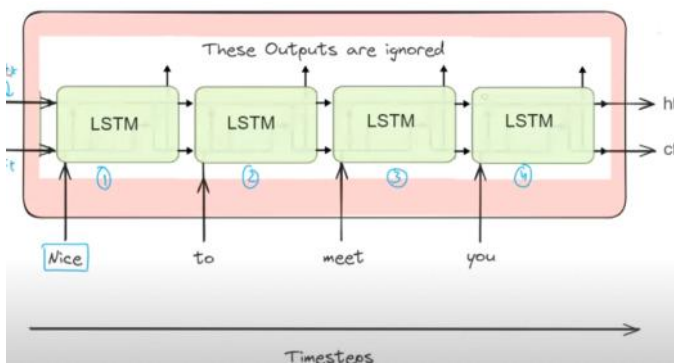
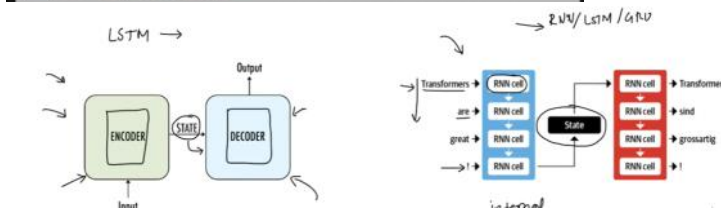
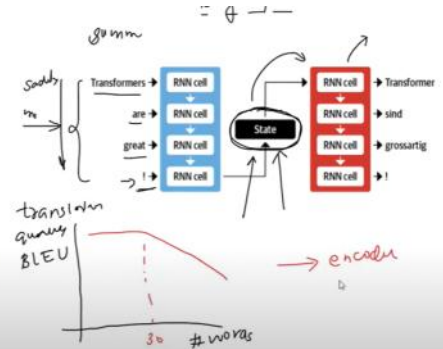
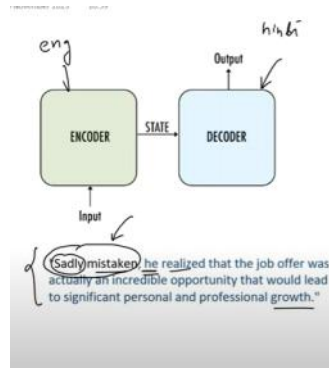
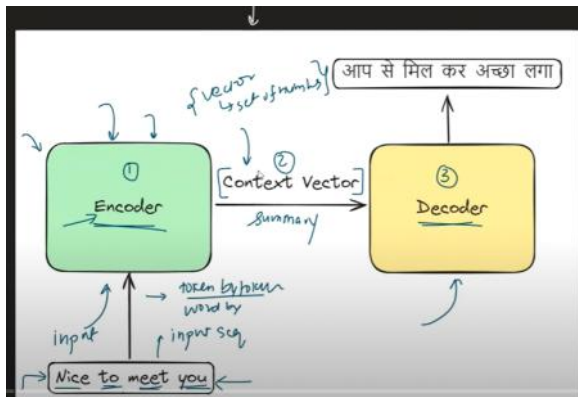
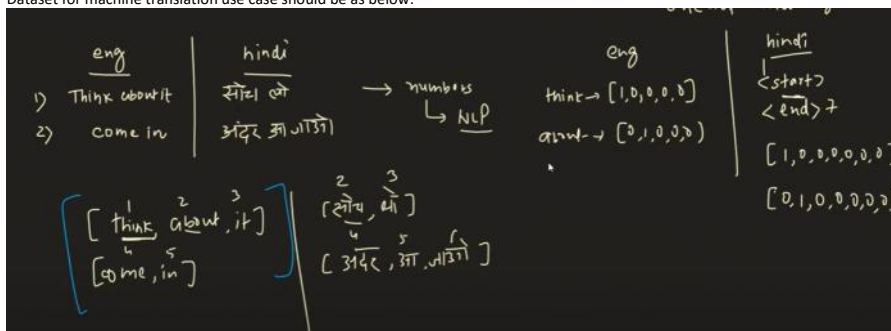


Encoder & Decoder

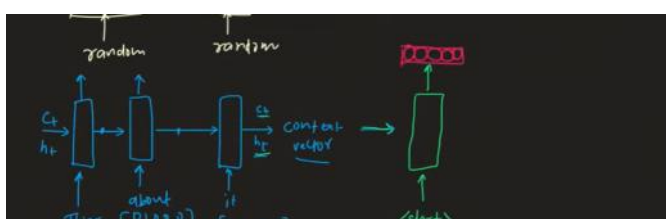
30 May 2024 07:33

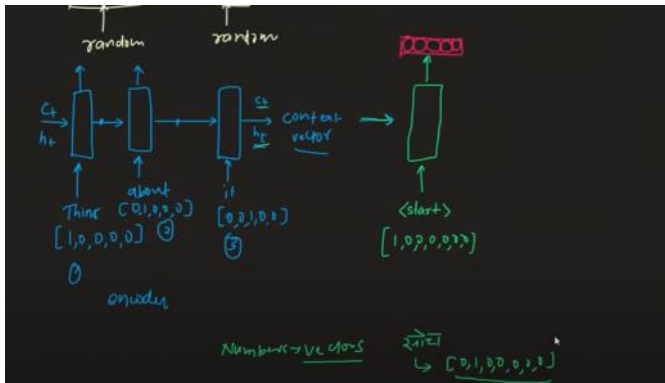


Dataset for machine translation use case should be as below:

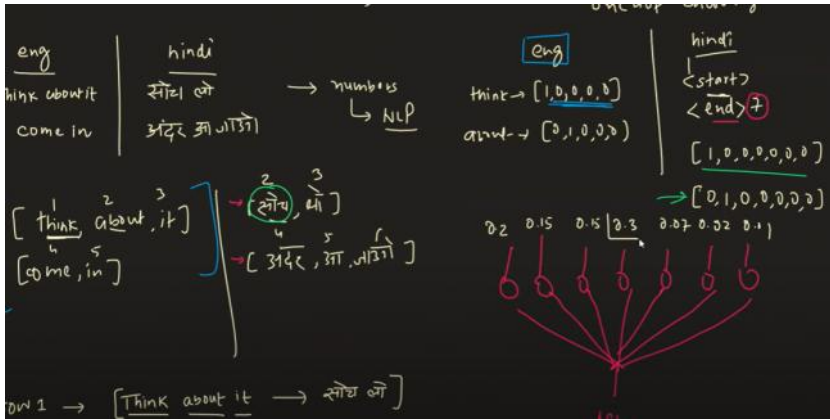


Training of the machine translation as below:

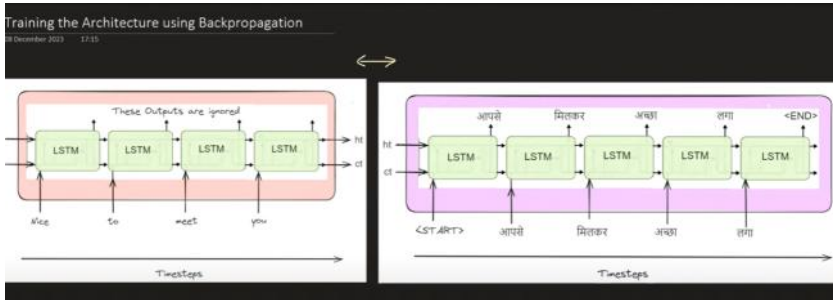




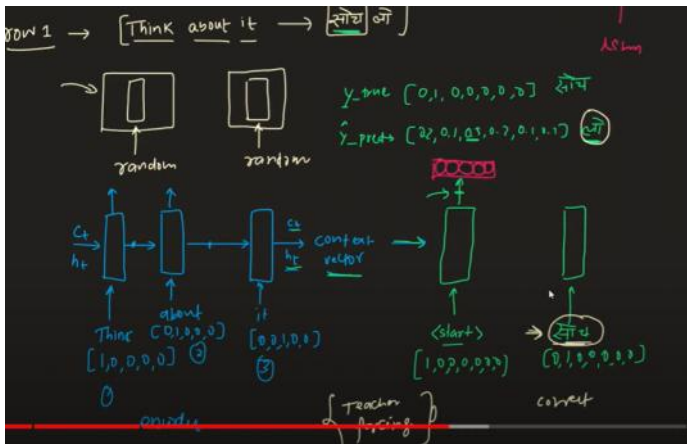
There is a softmax layer in the decoder and the number of nodes will be equal to total number of the translated words



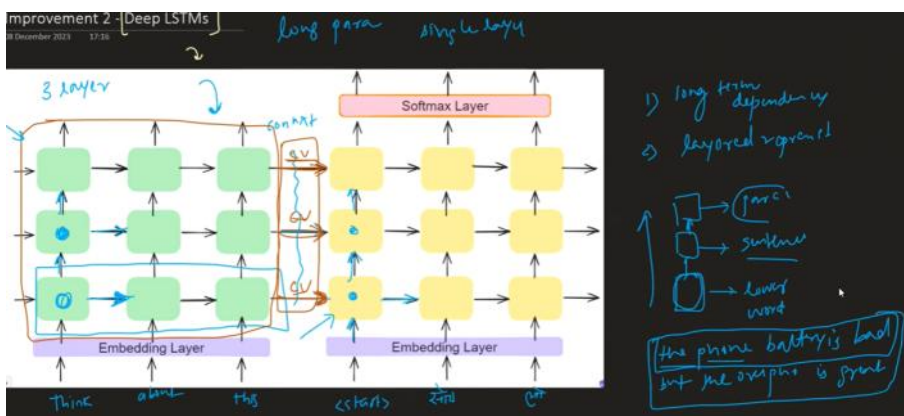
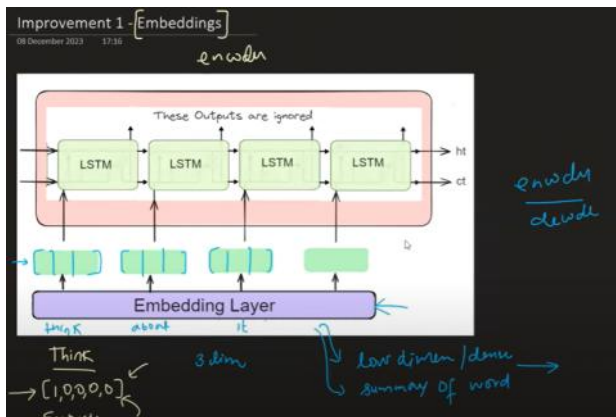
Encoder



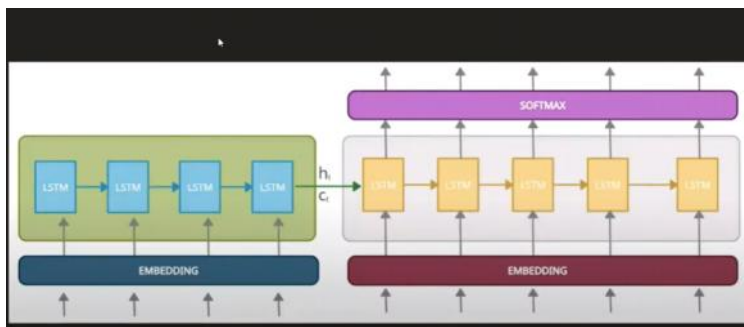
Decoder



If the probability of the word coming out from the decoder after 1st timestep is wrong, i.e. $\hat{y}_t$ is not equal to y_t , in that case also basis the translated dataset we will send the correct translated word as input to the decoder in the next timestep. This technique is called **teacher forcing**. Because of this the training will be faster.



Multiple layers of LSTM in encoder and decoder should be used in case large paragraphs. It works better for long paragraphs, also understands the hierarchy of the data (words -> sentence -> paragraph)



The Sutskever Architecture

Application to Translation: The model focused on translating English to French, demonstrating the effectiveness of sequence-to-sequence learning in neural machine translation.

Special End-of-Sentence Symbol: Each sentence in the dataset was terminated with a unique end-of-sentence symbol ("**<EOS>**"), enabling the model to recognize the end of a sequence.

Dataset: The model was trained on a subset of 12 million sentences, comprising 348 million French words and 304 million English words, taken from a publicly available dataset.

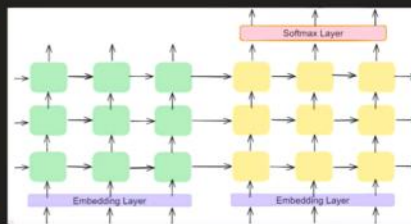
Vocabulary Limitation: To manage computational complexity, fixed vocabularies for both languages were used, with 160,000 most frequent words for English and 80,000 for French. Words not in these vocabularies were replaced with a special "UNK" token.

Reversing Input Sequences: The input sentences (English) were reversed before feeding them into the model, which was found to significantly improve the model's learning efficiency, especially for longer sentences.

Word Embeddings: The model used a 1000-dimensional word embedding layer to represent input words, providing dense, meaningful representations of each word.

Architecture Details: Both the input (encoder) and output (decoder) models had 4 layers, with each layer containing 1,000 units, showcasing a deep LSTM-based architecture.

Output Layer and Training: The output layer employed a Softmax function to generate the



end-of-sentence symbol ("**<EOS>**"), enabling the model to recognize the end of a sequence.

Dataset: The model was trained on a subset of **12 million sentences**, comprising **348 million French words** and **304 million English words**, taken from a publicly available dataset.

Vocabulary Limitation: To manage computational complexity, fixed vocabularies for both languages were used, with **160,000 most frequent words** for English and **80,000** for French. Words not in these vocabularies were replaced with a special "**UNK**" token.

Reversing Input Sequences: The input sentences (English) were reversed before feeding them into the model, which was found to significantly improve the model's learning efficiency, especially for longer sentences.

Word Embeddings: The model used a **1000-dimensional word embedding layer** to represent input words, providing dense, meaningful representations of each word.

Architecture Details: Both the input (encoder) and output (decoder) models had **4 layers**, with each layer containing **1,000 units**, showcasing a deep LSTM-based architecture.

Output Layer and Training: The output layer employed a Softmax function to generate the probability distribution over the target vocabulary. The model was trained end-to-end with these settings.

Performance - BLEU Score: The model achieved a **BLEU score of 34.81**, surpassing the baseline Statistical Machine Translation (SMT) system's score of **33.30** on the same dataset, marking a significant advancement in neural machine translation.

In an LSTM-based **encoder-decoder** model for **language translation** (e.g., English to Hindi):

- During **training**, we feed:
 - English sentence → **Encoder**
 - Corresponding Hindi sentence → **Decoder**

So your question is:

? At inference time, when we input a **new English phrase not seen during training**, how does the model still generate the correct Hindi translation?

Even though the **exact English phrase** wasn't in the training set, the model can still **generalize** because:

1. The encoder doesn't memorize phrases — it **learns patterns and structures** of language and encodes the **meaning** of the input.
2. The decoder learns to **map these meanings to Hindi sequences**, using what it has seen during training.
3. This is made possible by:
 - **Word embeddings** (similar words are mapped closely)
 - **LSTM's sequential memory**
 - **Exposure to many similar patterns during training**
 - (Optionally) **Attention mechanisms** to improve focus on relevant words

🔄 Inference Flow (Step-by-Step)

Let's say you trained your model on:

- "How are you?" → "तुम कैसे हो?"
- "I am fine" → "मैं ठीक हूँ"

Now at inference:

You input: "**How is your health?**" (unseen)

What happens:

1. Encoder:

- Tokenizes: ["how", "is", "your", "health"]
- Converts to embeddings (vectors representing meaning).
- Passes through the **encoder LSTM**, producing:
 - Final **hidden and cell states** (context vector).

2. Decoder:

- Starts with **<start>** token (e.g., **<sos>** or **"<BOS>"**).
- Uses encoder's context vector to initialize its LSTM state.
- Predicts next token: e.g., **"तुम"**
- Feeds **"तुम"** back into the decoder for the next step.
- Repeats until it outputs **<end>** or reaches max length.

🤔 But How Does It "Know" What to Translate?

Because during training:

🤔 But How Does It "Know" What to Translate?

Because during training:

- It learned how English grammar maps to Hindi grammar
- It learned semantic relationships (e.g., "health" → "स्वास्थ्य")
- It **generalized** across many similar sentences

So even if "How is your health?" was not in the training set,

- The model has seen "How is..." and "your health" before, possibly in different contexts
- It **composes a new response** based on **learned structures**